

## Kontener std::vector

### 1. Wiadomości wstępne

Kontener **std::vector** jest to, w wielkim uproszczeniu „inteligentna tablica”, która posiada przydatne funkcjonalności, jak np.: zna swój rozmiar, można łatwo dodawać/usuwać elementy z dowolnego miejsca tablicy, zmieniać rozmiar tablicy (w trakcie działania programu), posiada ciekawe metody np.: wyszukiwanie elementu, itp.. Cechuje się prostotą (elementów używa się jak w zwykłej tablicy), wygodą (dzięki dużej liczbie wbudowanych metod) i efektywności (np. nie musimy ręcznie zarządzać pamięcią przy np. zmianie rozmiaru).

Ten kto pozna vectory w C++, ten już (raczej) nigdy nie wróci do zwykłych tablic. :)

Aby móc skorzystać z **std::vector** należy zainkludować na początku bibliotekę: `#include <vector>`

Deklaracja vectora jak poniżej. Domyślnie tworzymy kontener vector o rozmiarze 0 elementów.

```
std::vector< typ_zmiennej > nazwa_vectora;
```

Przykładowo tworzymy **vector**, tablicę liczb całkowitych:

```
std::vector< int > vTab;
```

Sprawdzenie rozmiaru kontenera (ile ma elementów).

```
int rozmiar = vTab.size();
```

Dodanie na koniec vectora (zawsze będzie to jako nowy ostatni element) jednego elementu (w przykładzie liczba **15**). Wszystkie elementy muszą być tylko takiego typu, jakiego jest vector zadeklarowany. Rozmiar vectora zwiększy się o 1.

```
vTab.push_back( 15 );
```

Zwracamy elementy tak, jakby były to elementy ze zwykłej tablicy.

Przykładowo sprawdzamy jaka jest wartość elementu o indeksie zero.

```
int zmienna = vTab[ 0 ];
```

Ustawiamy elementy tak, jakby były to elementy w zwykłej tablicy. Przykładowo ustawiamy wartość elementu o indeksie zero na **999**.

```
vTab[0] = 999;
```

Usunięcie ostatniego elementu (z końca vectora). Rozmiar vectora zmniejszy się o 1.

```
vTab.pop_back();
```

Kontener `std::vector`

## 2. vector jako parametr funkcji

Aby ułatwić sobie pracę z vectorami można stworzyć funkcje i przekazać vector jako parametr. Tym sposobem, na przykład, raz napisana funkcja drukująca będzie mogła być po prostu wywoływana, bez konieczności kopiowania całego kodu pętli odpowiedzialnej za wyświetlanie (oszczędność czasu i miejsca).

**Odradza się** przekazywanie vectora jako **parametr w postaci kopii wartości zmiennej**, bo niepotrzebnie tworzy to jego kopię (miejsce w pamięci!), a vectory mogą być bardzo bardzo duże. No i zmiany naniesione na kopii nie będą miały efektu modyfikacji na elementach oryginalnego vectora.

Znacznie lepszym rozwiązaniem jest przekazanie vectora jako **parametru w postaci referencji** (ewentualnie **wskaźnika**, jeśli ktoś woli). Referencja, to tylko adres pamięci, także wynosi on zawsze tyle samo, niezależnie czy vector ma 10, czy 10000000 elementów.

(**Dobra praktyka**) Jeśli vector ma być w funkcji tylko do odczytu, to blokujemy deklarację parametru modyfikatorem **const**.

(Przypomnij sobie jak działały funkcje z parametrami różnych typów. Zerknij do materiałów - ćwiczenia z poprzednich zajęć. Jak nie chciało Ci się zrobić zadania, to teraz masz problem. ;) ).

Przykładowo, deklaracja vectora w głównym kodzie:

```
std::vector< int > vTab;
```

Deklaracja parametru funkcji jako referencji (**&**) na vector. Modyfikator **const**, bo funkcja ma drukować, więc zabezpieczamy sobie vector przed (przypadkową) modyfikacją elementów.

```
void Funkcja_drukuj( const std::vector< int > &vRef ) { // Tutaj kod funkcji. Parametru (referencji) używamy tak, // jakby to był zwykły vector zadeklarowany w tej funkcji, // (choć naprawdę odwołujemy się do vectora oryginalnego)
```

Wywołanie funkcji z parametrem w głównym kodzie:  
Funkcja\_drukuj( vTab );

Kontener std::vector

### 3. Wskaźniki – jeśli ktoś preferuje...

Jeśli ktoś nie lubi używać referencji, to może także przekazać vectora do funkcji w postaci wskaźnika. Jednak jest to zapis bardziej kłopotliwy.

Przykładowo, deklaracja vectora w głównym kodzie:  
std::vector< int > vTab;

Deklaracja parametru funkcji jako wskaźnika (\*) na vector. Modyfikator **const**, bo funkcja ma drukować, więc zabezpieczamy sobie vector przed (przypadkową) modyfikacją elementów.

```
void Funkcja_drukuj( const std::vector< int > *vT ) { // Tutaj kod funkcji. Parametru  
(wskaźnika) używamy, lecz // naprawdę odwołujemy się do vectora oryginalnego i to na  
// nim wykonujemy operacje
```

Przykładowe operacje na vectorze wskaźniku zadeklarowanym jak powyżej.  
int n = (\*vT)[ 0 ];  
(\*vT)[ 0 ] = 999;

Wywołanie funkcji z parametrem w głównym kodzie. Wskazujemy, że do funkcji przekazujemy **adres pamięci**, gdzie się znajduje vector.  
Funkcja\_drukuj( &vTab );